Assignment-1

Vijaya Sreyas Badde
AI24BTECH11003

AI2100
Deep Learning

---

1 Artificial Neural Network with Backpropagation

1.1 Network Architecture

A simple feedforward neural network was implemented from scratch using NumPy. The network consists of two input nodes, one hidden layer with two neurons, and one output node.

The forward pass of the network is defined as:

$$z^{(1)} = W^{(1)}x + b^{(1)}, \qquad a^{(1)} = \sigma(z^{(1)})$$

$$z^{(2)} = W^{(2)}a^{(1)} + b^{(2)}, \qquad \hat{y} = \sigma(z^{(2)})$$

where $\sigma(\cdot)$ denotes the sigmoid activation function.

The network parameters are trained using the backpropagation algorithm with gradient descent.

1.2 Training Setup

Two tasks were used to evaluate the neural network:

- Learning the XOR logical function
- Approximating the cosine function

For both tasks, datasets were split into training and testing sets using an 80%–20% split.

The loss function used was Mean Squared Error (MSE):

$$L = \frac{1}{N}\sum_i (y_i - \hat{y}_i)^2$$

Weights were initialized randomly and updated using gradient descent. Training was performed over multiple epochs using the backpropagation algorithm to compute gradients of the loss with respect to the network parameters.

Model performance was evaluated using both loss and classification accuracy (for the XOR task).

1.3 XOR Function Learning

The XOR function is a classical example of a problem that is not linearly separable. A neural network with a hidden layer can represent the required non-linear decision boundary.

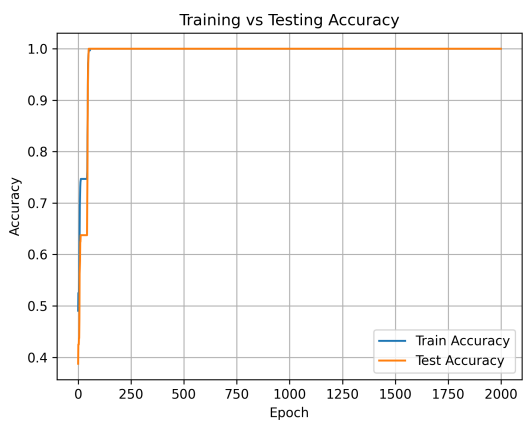
The XOR dataset was generated by sampling the four binary input combinations $(0, 0)$, $(0, 1)$, $(1, 0)$, and $(1, 1)$. To create a larger dataset and avoid perfectly discrete points, Gaussian noise was added to the input features:

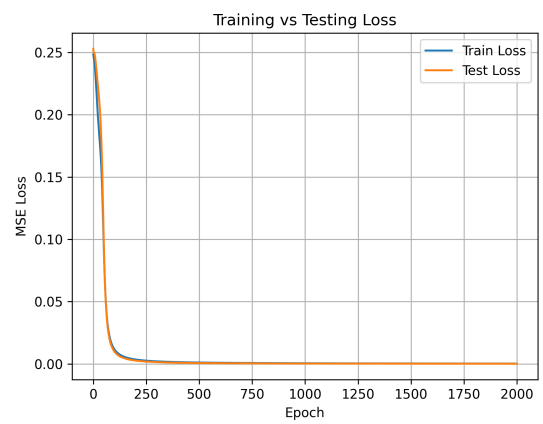
$$x' = x + \epsilon, \quad \epsilon \sim \mathcal{N}(0, 0.1^2)$$

This produces a slightly perturbed dataset while preserving the XOR decision structure.

Training results show that the network is able to correctly classify the XOR dataset with high accuracy after sufficient training iterations.

Training and testing loss curves demonstrate stable convergence during training.





1.4 Cosine Function Learning

The network was also trained to approximate the cosine function over the interval $[-\pi, \pi]$.

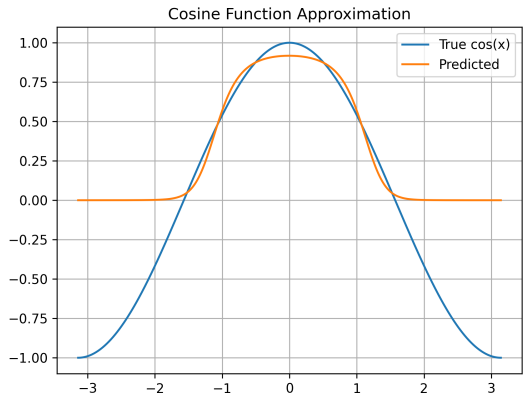
Since the network architecture requires two input nodes, but the cosine function depends on only a single variable, an additional dummy input was introduced. The input to the network was defined as

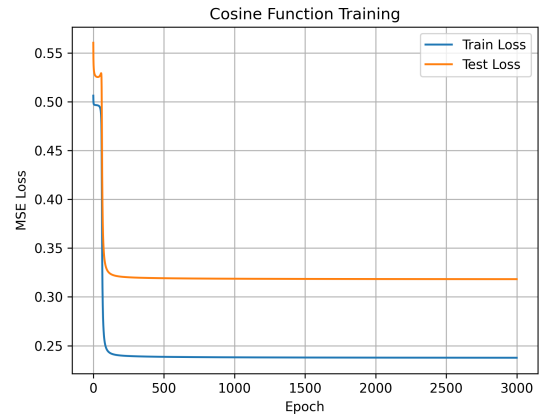
$$x_{input} = (x, 0)$$

where x is the sampled value from the interval $[-\pi, \pi]$ and the second input dimension is fixed at zero.

This approach preserves the required two–input network structure while still allowing the network to learn the mapping

$$(x, 0) \to \cos(x).$$





1.5 Discussion

These experiments demonstrate the ability of neural networks to learn both logical functions and continuous function approximations using backpropagation.

The XOR experiment illustrates how hidden layers enable networks to solve non-linear classification problems. The cosine approximation experiment demonstrates the function approximation capability of neural networks, even with a very small architecture.

2 Universal Approximation Theorem

(a) Formal Statement

Let $K \subset \mathbb{R}^n$ be a compact set and let $C(K)$ denote the space of continuous real-valued functions defined on K.

The Universal Approximation Theorem states that a feedforward neural network with a single hidden layer and a non-polynomial continuous activation function (such as the sigmoid function) can approximate any function in $C(K)$ arbitrarily well.

More formally, for any $f \in C(K)$ and any $\epsilon > 0$, there exists a neural network of the form

$$g(x) = \sum_{i=1}^{m} \alpha_i \, \sigma(w_i^T x + b_i)$$

such that

$$\sup_{x \in K} |f(x) - g(x)| < \epsilon.$$

Here $\sigma(\cdot)$ is a non-polynomial activation function, m is the number of hidden neurons, and the parameters α_i, w_i, and b_i are real values. This result implies that the class of neural networks with a single hidden layer is dense in $C(K)$ under the supremum norm.

(b) Proof Idea

The main idea behind the Universal Approximation Theorem is that neurons with non-linear activation functions can act as flexible basis functions. By combining many such functions, a neural network can construct complex shapes that approximate arbitrary continuous functions.

In particular, functions of the form $\sigma(w^T x + b)$ can be used to approximate localized or step-like functions over the input domain. By combining many such localized responses, the network can approximate more complex functions.

This idea is closely related to classical approximation results such as the Stone–Weierstrass theorem, which states that certain families of functions are dense in the space of continuous functions on a compact domain.

(c) Limitations

Although the Universal Approximation Theorem guarantees representational capacity, it has several limitations in practical machine learning.

First, the theorem does not specify how many neurons are required. In many cases, approximating a complex function with a single hidden layer may require a very large number of neurons, making the model inefficient.

Second, the theorem provides only an existence result and does not guarantee that the required parameters can be found using practical training methods. Training neural networks involves solving a highly non-convex optimization problem, and gradient-based methods may converge slowly or become trapped in poor local minima.

(d) ReLU and Depth

The universal approximation property also holds when using ReLU activation functions. However, the role of network depth becomes particularly important.

While a single hidden layer network can approximate any continuous function, it may require an extremely large number of neurons. Deep networks with multiple hidden layers can often represent the same functions using far fewer parameters.

Increasing depth allows networks to compose simple nonlinear transformations, leading to more efficient representations of complex functions. This is one of the key motivations for the use of deep architectures in modern neural networks.

3 One Step of Gradient Descent

3.1 Network Architecture

We consider a feedforward neural network with three affine layers and ReLU activation functions in the hidden layers.

The input is

$$x = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \in \mathbb{R}^2.$$

The first hidden layer computes

$$z^{(1)} = W^{(1)}x + b^{(1)}, \quad a^{(1)} = \mathrm{ReLU}(z^{(1)})$$

where

$$W^{(1)} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad b^{(1)} = \begin{bmatrix} 0 & 0 \end{bmatrix}.$$

The second hidden layer computes

$$z^{(2)} = W^{(2)}a^{(1)} + b^{(2)}, \quad a^{(2)} = \mathrm{ReLU}(z^{(2)})$$

with

$$W^{(2)} = \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}, \quad b^{(2)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

The output layer computes the logits

$$s = W^{(3)}a^{(2)} + b^{(3)}$$

where

$$W^{(3)} = \begin{bmatrix} 1 & 0 \\ & 1 \end{bmatrix}, \quad b^{(3)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

The true class label is $y = 1$ and the learning rate is $\eta = 0.1$.

3.2 Part A: Softmax with Cross-Entropy Loss

(a) Forward Pass

The input vector is

$$x = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

Layer 1

$$z^{(1)} = W^{(1)}x + b^{(1)} \tag{1}$$

$$= \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \tag{2}$$

Applying the ReLU activation,

$$a^{(1)} = \mathrm{ReLU}(z^{(1)}) = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

Layer 2

$$z^{(2)} = W^{(2)}a^{(1)} + b^{(2)} \tag{3}$$

$$= \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 2 \end{bmatrix} \tag{4}$$

Applying ReLU again,

$$a^{(2)} = \begin{bmatrix} 0 \\ 2 \end{bmatrix}.$$

Output Layer

$$s = W^{(3)}a^{(2)} + b^{(3)} \tag{5}$$

$$= \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 2 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 2 \end{bmatrix}. \tag{6}$$

Softmax Probabilities

The softmax function is

$$q_i = \frac{e^{s_i}}{\sum_j e^{s_j}}.$$

Since

$$e^0 = 1, \qquad e^2 \approx 7.389,$$

the denominator becomes

$$1 + 7.389 = 8.389.$$

Thus the softmax probabilities are

$$q_1 = \frac{1}{8.389} \approx 0.119 \tag{7}$$

$$q_2 = \frac{7.389}{8.389} \approx 0.881 \tag{8}$$

$$q = \begin{bmatrix} 0.119 \\ 0.881 \end{bmatrix} \tag{9}$$

(b) Cross-Entropy Loss

The cross-entropy loss for a single example is

$$L_{CE} = -\log q_y.$$

Since the true class label is $y = 1$,

$$L_{CE} = -\log(0.119) \approx 2.13.$$

(c) Gradient with Respect to Logits

For softmax combined with cross-entropy loss, the gradient with respect to the logits is

$$\frac{\partial L}{\partial s} = q - y.$$

The one-hot representation of the true label $y = 1$ is

$$y = \begin{bmatrix} 1 \\ 0 \end{bmatrix}.$$

Thus,

$$\nabla_s L = \begin{bmatrix} 0.119 \\ 0.881 \end{bmatrix} - \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} -0.881 \\ 0.881 \end{bmatrix}.$$

(d) Gradients via Backpropagation

We compute gradients for all parameters using backpropagation.

Output Layer

The gradient with respect to the output weights is

$$\nabla_{W^{(3)}} L = (\nabla_s L)(a^{(2)})^T \tag{10}$$

Substituting the computed values

$$\nabla_s L = \begin{bmatrix} -0.881 \\ 0.881 \end{bmatrix}, \quad a^{(2)} = \begin{bmatrix} 0 \\ 2 \end{bmatrix} \tag{11}$$

$$\nabla_{W^{(3)}} L = \begin{bmatrix} -0.881 \\ 0.881 \end{bmatrix} \begin{bmatrix} 0 & 2 \end{bmatrix} = \begin{bmatrix} 0 & -1.762 \\ 0 & 1.762 \end{bmatrix} \tag{12}$$

The bias gradient is

$$\nabla_{b^{(3)}} L = \begin{bmatrix} -0.881 \\ 0.881 \end{bmatrix}. \tag{13}$$

Layer 2

First we propagate the gradient to the hidden activations:

$$\nabla_{a^{(2)}} L = (W^{(3)})^T \nabla_s L = \begin{bmatrix} -0.881 \\ 0.881 \end{bmatrix} \tag{14}$$

Applying the derivative of ReLU,

$$\nabla_{z^{(2)}} L = \nabla_{a^{(2)}} L \odot \mathrm{ReLU}'(z^{(2)}) \tag{15}$$

Since

$$z^{(2)} = \begin{bmatrix} 0 \\ 2 \end{bmatrix} \tag{16}$$

the derivative becomes

$$\nabla_{z^{(2)}} L = \begin{bmatrix} 0 \\ 0.881 \end{bmatrix}. \tag{17}$$

Now compute the gradients:

$$\nabla_{W^{(2)}} L = (\nabla_{z^{(2)}} L)(a^{(1)})^T \tag{18}$$

$$= \begin{bmatrix} 0 \\ 0.881 \end{bmatrix} \begin{bmatrix} 1 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 0.881 & 0.881 \end{bmatrix} \tag{19}$$

$$\nabla_{b^{(2)}} L = \begin{bmatrix} 0 \\ 0.881 \end{bmatrix}. \tag{20}$$

Layer 1
Propagating further,

$$\nabla_{a^{(1)}} L = (W^{(2)})^T \nabla_{z^{(2)}} L \tag{21}$$

$$= \begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0.881 \end{bmatrix} = \begin{bmatrix} 0.881 \\ 0.881 \end{bmatrix} \tag{22}$$

Since $z^{(1)} > 0$, the ReLU derivative is 1, giving

$$\nabla_{z^{(1)}} L = \begin{bmatrix} 0.881 \\ 0.881 \end{bmatrix}. \tag{23}$$

Finally,

$$\nabla_{W^{(1)}} L = (\nabla_{z^{(1)}} L) x^T \tag{24}$$

$$= \begin{bmatrix} 0.881 \\ 0.881 \end{bmatrix} \begin{bmatrix} 1 & 1 \end{bmatrix} = \begin{bmatrix} 0.881 & 0.881 \\ 0.881 & 0.881 \end{bmatrix} \tag{25}$$

$$\nabla_{b^{(1)}} L = \begin{bmatrix} 0.881 \\ 0.881 \end{bmatrix}. \tag{26}$$

(e) Gradient Descent Update

Using gradient descent with learning rate $\eta = 0.1$,

$$\theta_{\text{new}} = \theta - \eta \nabla_\theta L \tag{27}$$

Output Layer

$$W^{(3)}_{\text{new}} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} - 0.1 \begin{bmatrix} 0 & -1.762 \\ 0 & 1.762 \end{bmatrix} = \begin{bmatrix} 1 & 0.1762 \\ 0 & 0.8238 \end{bmatrix} \tag{28}$$

$$b^{(3)}_{\text{new}} = \begin{bmatrix} 0.0881 \\ -0.0881 \end{bmatrix} \tag{29}$$

Layer 2

$$W^{(2)}_{\text{new}} = \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix} - 0.1 \begin{bmatrix} 0 & 0 \\ 0.881 & 0.881 \end{bmatrix} = \begin{bmatrix} 1 & -1 \\ 0.9119 & 0.9119 \end{bmatrix} \tag{30}$$

$$b^{(2)}_{\text{new}} = \begin{bmatrix} 0 \\ -0.0881 \end{bmatrix} \tag{31}$$

Layer 1

$$W^{(1)}_{\text{new}} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} - 0.1 \begin{bmatrix} 0.881 & 0.881 \\ 0.881 & 0.881 \end{bmatrix} = \begin{bmatrix} 0.9119 & -0.0881 \\ -0.0881 & 0.9119 \end{bmatrix} \tag{32}$$

$$b^{(1)}_{\text{new}} = \begin{bmatrix} -0.0881 \\ -0.0881 \end{bmatrix} \tag{33}$$

3.3 Part B: Multiclass Hinge Loss

(a) Hinge Loss

The multiclass hinge loss is defined as

$$L_{\text{hinge}} = \sum_{j \neq y} \max(0, s_j - s_y + 1) \tag{34}$$

Using the previously computed logits

$$s = \begin{bmatrix} 0 \\ 2 \end{bmatrix} \tag{35}$$

and the true label $y = 1$,

$$L_{\text{hinge}} = \max(0, s_2 - s_1 + 1) \tag{36}$$
$$= \max(0, 2 - 0 + 1) \tag{37}$$
$$= 3 \tag{38}$$

(b) Gradient with Respect to Logits

For hinge loss,

$$\frac{\partial L}{\partial s_j} = \begin{cases} 1 & s_j - s_y + 1 > 0 \\ 0 & \text{otherwise} \end{cases} \tag{39}$$

Since $s_2 - s_1 + 1 = 3 > 0$,

$$\nabla_s L = \begin{bmatrix} -1 \\ 1 \end{bmatrix} \tag{40}$$

(c) Gradients via Backpropagation

Output Layer

$$\nabla_{W^{(3)}} L = (\nabla_s L)(a^{(2)})^T \tag{41}$$
$$= \begin{bmatrix} -1 \\ 1 \end{bmatrix} \begin{bmatrix} 0 & 2 \end{bmatrix} \tag{42}$$
$$= \begin{bmatrix} 0 & -2 \\ 0 & 2 \end{bmatrix} \tag{43}$$

$$\nabla_{b^{(3)}} L = \begin{bmatrix} -1 \\ 1 \end{bmatrix} \tag{44}$$

Layer 2

$$\nabla_{a^{(2)}} L = (W^{(3)})^T \nabla_s L = \begin{bmatrix} -1 \\ 1 \end{bmatrix} \tag{45}$$

Applying ReLU derivative,

$$\nabla_{z^{(2)}} L = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \tag{46}$$

$$\nabla_{W^{(2)}} L = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 1 \end{bmatrix} \tag{47}$$

$$= \begin{bmatrix} 0 & 0 \\ 1 & 1 \end{bmatrix} \tag{48}$$

$$\nabla_{b^{(2)}} L = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \tag{49}$$

Layer 1

$$\nabla_{a^{(1)}} L = (W^{(2)})^T \nabla_{z^{(2)}} L \tag{50}$$

$$= \begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} \tag{51}$$

$$= \begin{bmatrix} 1 \\ 1 \end{bmatrix} \tag{52}$$

$$\nabla_{W^{(1)}} L = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 1 \end{bmatrix} \tag{53}$$

$$= \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \tag{54}$$

$$\nabla_{b^{(1)}} L = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \tag{55}$$

(d) Gradient Descent Update

Using learning rate $\eta = 0.1$,

$$W^{(3)}_{\text{new}} = \begin{bmatrix} 1 & 0.2 \\ 0 & 0.8 \end{bmatrix} \tag{56}$$

$$b^{(3)}_{\text{new}} = \begin{bmatrix} 0.1 \\ -0.1 \end{bmatrix} \tag{57}$$

$$W_{\text{new}}^{(2)} = \begin{bmatrix} 1 & -1 \\ 0.9 & 0.9 \end{bmatrix} \tag{58}$$

$$b_{\text{new}}^{(2)} = \begin{bmatrix} 0 \\ -0.1 \end{bmatrix} \tag{59}$$

$$W_{\text{new}}^{(1)} = \begin{bmatrix} 0.9 & -0.1 \\ -0.1 & 0.9 \end{bmatrix} \tag{60}$$

$$b_{\text{new}}^{(1)} = \begin{bmatrix} -0.1 \\ -0.1 \end{bmatrix} \tag{61}$$

These updated parameters represent the result of one gradient descent step performed on the network for the given input example using the specified loss function.

5 ADALINE

5.1 Theory

The ADALINE (Adaptive Linear Neuron) model was introduced by Widrow and Hoff (1960) as one of the earliest learning algorithms for neural systems. Unlike the perceptron, which updates weights based on classification mistakes, ADALINE performs learning by minimizing a continuous objective function using gradient descent.

The model predicts an output using a linear function

$$\hat{y} = w^T x + b \tag{62}$$

where w is the weight vector and b is the bias.

Training minimizes the mean squared error (MSE)

$$L(w, b) = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2. \tag{63}$$

The gradient of the loss with respect to the parameters is

$$\frac{\partial L}{\partial w} = -\frac{2}{N} X^T (y - \hat{y}) \tag{64}$$

$$\frac{\partial L}{\partial b} = -\frac{2}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i). \tag{65}$$

Using gradient descent, the parameters are updated according to the Widrow–Hoff learning rule

$$w \leftarrow w - \eta \frac{\partial L}{\partial w} \tag{66}$$

$$b \leftarrow b - \eta \frac{\partial L}{\partial b} \tag{67}$$

where η is the learning rate.

5.2 Convergence to the Least Squares Solution

The ADALINE objective is a quadratic function of the parameters. Writing the loss in matrix form,

$$L(w) = \frac{1}{N} \|Xw - y\|^2, \tag{68}$$

where X is the data matrix and y is the target vector.

This objective is convex and has a unique global minimum given by the least-squares solution

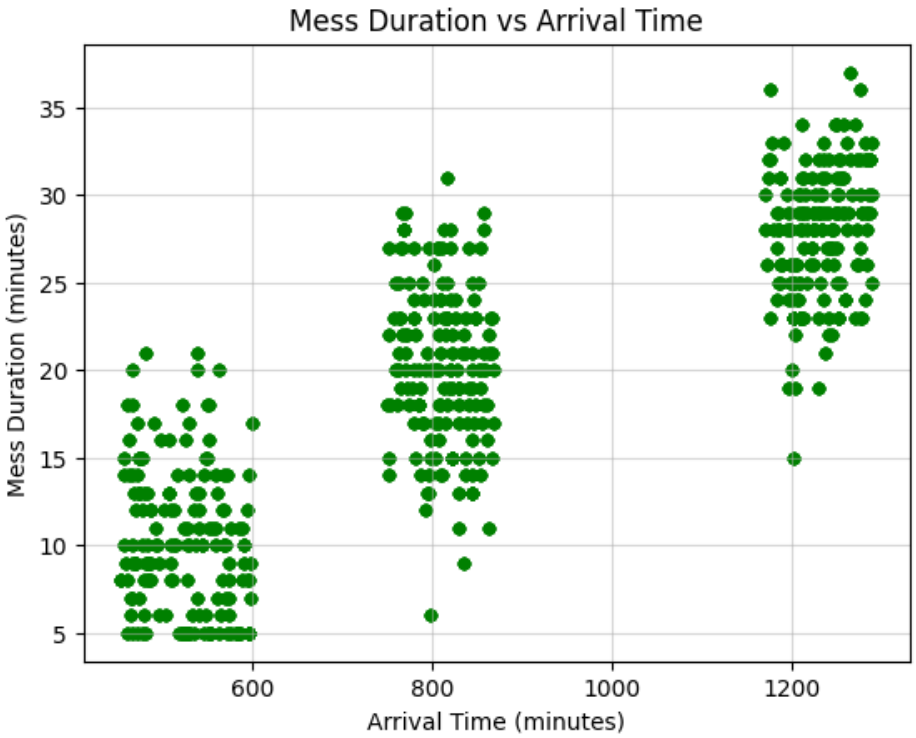
$$w^* = (X^T X)^{-1} X^T y. \tag{69}$$

Because the loss function is convex, gradient descent is guaranteed to converge to this global minimum for sufficiently small learning rates. Importantly, this convergence does not depend on the data being linearly separable. As a result, ADALINE can be used for regression problems even when perfect separation is impossible.

5.3 Experiments

(a) Dataset Visualization

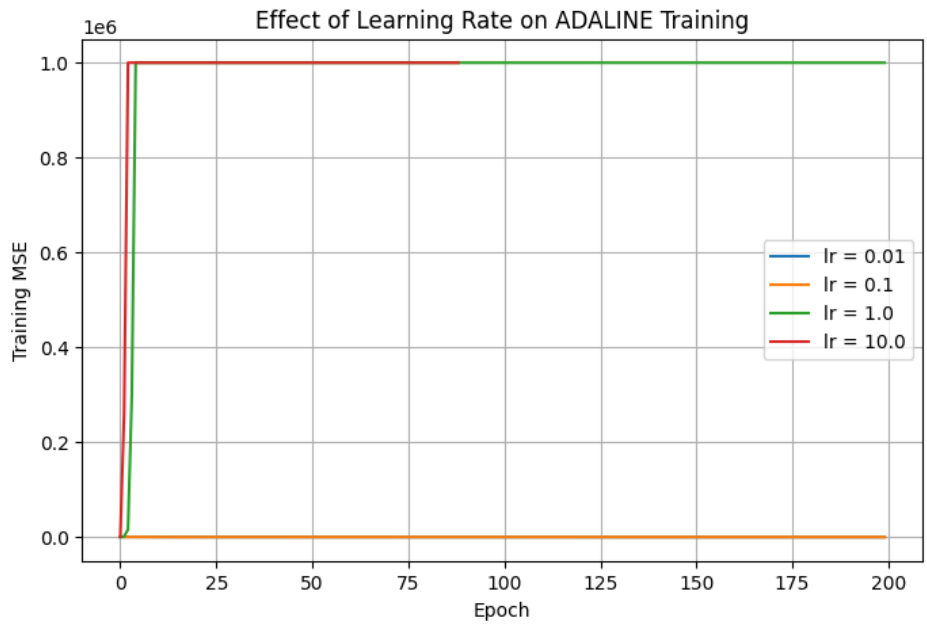
We visualize the relationship between arrival time and mess duration. The plot reveals three clusters corresponding to breakfast, lunch, and dinner time windows. This structure reflects the simulated campus mess schedule and provides insight into the relationship between arrival time and expected duration of stay.



(b) Effect of Learning Rate

We evaluate the effect of different learning rates $\eta \in \{0.01, 0.1, 1.0, 10.0\}$ on the convergence behavior of ADALINE.

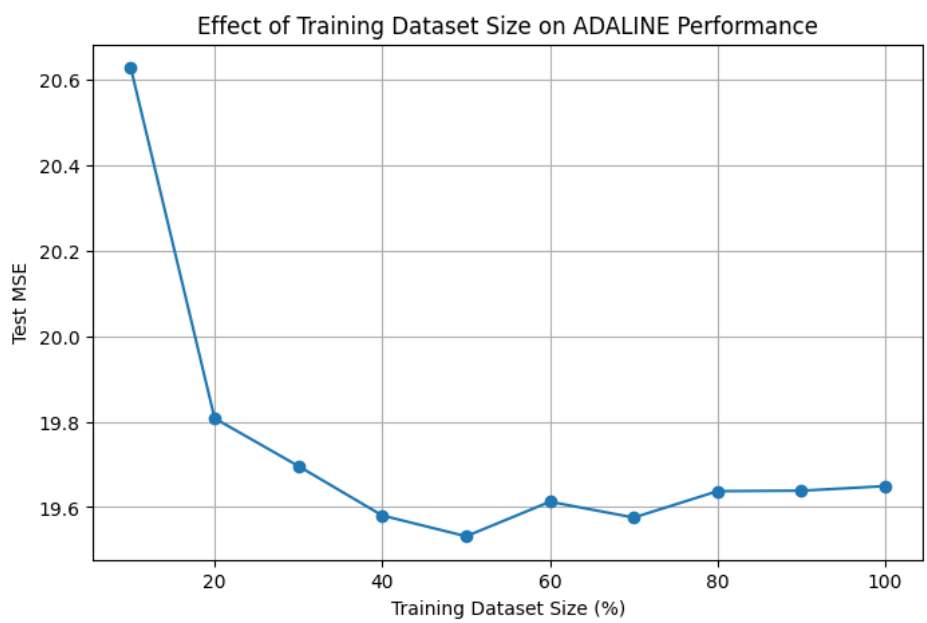
Small learning rates produce slow but stable convergence. Moderate learning rates converge faster. However, very large learning rates lead to unstable updates and divergence, as the gradient descent steps overshoot the minimum of the loss function. This behavior illustrates the importance of choosing an appropriate learning rate when training models with gradient-based optimization.



(c) Effect of Training Dataset Size

To study the impact of dataset size, we train ADALINE using subsets of the training data ranging from 10% to 100% in increments of 10%.

The results show that the test mean squared error decreases as the amount of training data increases. The largest improvement occurs when increasing the dataset from very small sizes to moderate sizes. Beyond roughly 70–80% of the data, the performance improvements become smaller, indicating diminishing returns from additional data. Minor fluctuations are expected due to random sampling of training subsets.



6 MLP with Backpropagation

6.1 Theory

(a) Vanishing Gradient Problem

Consider a deep neural network with L layers. For a given layer l, the forward computation is

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)} \tag{70}$$

$$a^{(l)} = \sigma(z^{(l)}) \tag{71}$$

where $\sigma(\cdot)$ is the activation function.

Using backpropagation, the gradient of the loss L with respect to the weights of layer l is

$$\frac{\partial L}{\partial W^{(l)}} = \frac{\partial L}{\partial a^{(L)}} \prod_{k=l}^{L-1} \left(W^{(k+1)} \sigma'(z^{(k)}) \right). \tag{72}$$

This expression shows that gradients propagate through the network as a product of many terms involving activation derivatives and weight matrices.

For activation functions such as the sigmoid,

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)), \tag{73}$$

and the derivative satisfies

$$0 < \sigma'(z) \leq 0.25. \tag{74}$$

Thus, during backpropagation the gradient is repeatedly multiplied by numbers smaller than one. In deep networks this causes the gradient magnitude to shrink exponentially with depth,

$$\left\| \frac{\partial L}{\partial W^{(l)}} \right\| \to 0 \quad \text{as } L \text{ increases.} \tag{75}$$

As a result, the early layers receive extremely small gradients and learn very slowly. This phenomenon is known as the **vanishing gradient problem**.

Activation functions such as ReLU alleviate this issue because their derivative is

$$\mathrm{ReLU}'(z) = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases} \tag{76}$$

which avoids repeatedly multiplying gradients by small values.

(b) Gradient Propagation in Deep Networks

During backpropagation, gradients are propagated through multiple layers using the chain rule. For a network with L layers, the gradient with respect to the parameters of layer l involves a product of derivatives from all subsequent layers,

$$\frac{\partial L}{\partial W^{(l)}} = \frac{\partial L}{\partial a^{(L)}} \prod_{k=l}^{L-1} \left(W^{(k+1)} \sigma'(z^{(k)}) \right). \tag{77}$$

Thus, the gradient magnitude depends on the product of several terms involving activation derivatives and weights.

If the expected magnitude of these terms is less than one, the expected gradient behaves approximately as

$$E\left[\left| \frac{\partial L}{\partial W^{(l)}} \right| \right] \propto c^{L-l}, \quad 0 < c < 1. \tag{78}$$

As the depth of the network increases, this exponential decay causes gradients in earlier layers to become extremely small. Consequently, the parameters in the early layers receive very weak updates during training.

This analysis explains why deep networks with saturating activation functions such as sigmoid or tanh often experience slow learning in early layers.

6.2 Implementation

A flexible Multi-Layer Perceptron (MLP) framework was implemented from scratch using NumPy. The implementation supports multiple hidden layers, configurable activation functions, several optimization algorithms, and optional regularization.

Let the network consist of L layers. For layer l, the affine transformation is

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}$$

where $W^{(l)}$ and $b^{(l)}$ denote the weight matrix and bias vector respectively.

The activation is then computed as

$$a^{(l)} = \sigma^{(l)}(z^{(l)})$$

where $\sigma^{(l)}$ is the activation function for layer l.

During training, gradients are computed using backpropagation and parameters are updated using different optimization algorithms. The framework supports mini-batch training and modular components for:

- Activation functions
- Loss functions

- Optimization algorithms
- Weight initialization
- Regularization

6.3 Experiments: Architecture Ablation

6.3.1 Depth Ablation

To study the effect of network depth, the hidden layer width was fixed at 64 neurons while varying the number of hidden layers.

The tested architectures were:

- 1 hidden layer: $[\text{input}, 64, \text{output}]$
- 2 hidden layers: $[\text{input}, 64, 64, \text{output}]$
- 3 hidden layers: $[\text{input}, 64, 64, 64, \text{output}]$
- 4 hidden layers: $[\text{input}, 64, 64, 64, 64, \text{output}]$

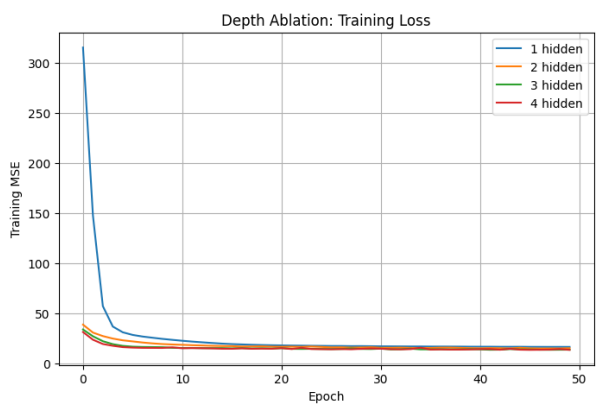


Figure 1: Training loss vs. epochs

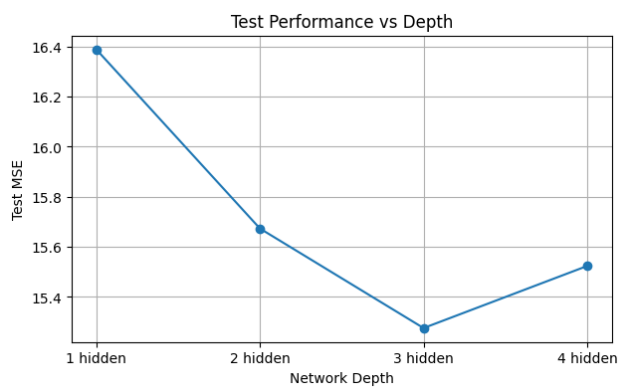


Figure 2: Test MSE vs. network depth

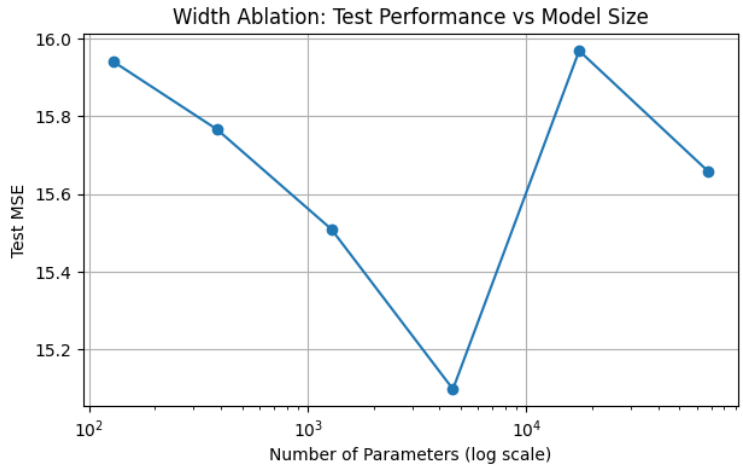
The results show that increasing depth initially improves performance, with the three-layer architecture achieving the lowest test error. However, deeper networks did not significantly improve performance further and may introduce additional optimization difficulty.

6.3.2 Width Ablation

To analyze the effect of network width, the depth was fixed at two hidden layers while varying the number of neurons per layer.

The tested architectures were:

$$[\text{input}, w, w, \text{output}], \quad w \in \{8, 16, 32, 64, 128, 256\}$$



Increasing the width generally improves the expressive power of the network, but after a certain point the improvement becomes marginal. Very large networks increase computational cost without providing significant performance gains.

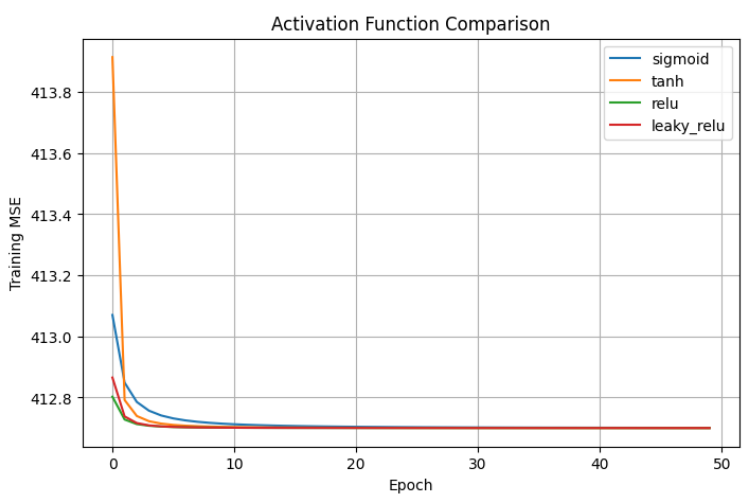
6.3.3 Activation Function Comparison

We compared several activation functions while keeping the architecture fixed as

$$[\text{input}, 64, 64, \text{output}]$$

The following configurations were tested:

- Sigmoid in all layers
- Tanh in all layers
- ReLU in hidden layers with sigmoid output
- Leaky ReLU in hidden layers with sigmoid output



Sigmoid networks exhibited slower convergence due to vanishing gradients. ReLU and Leaky ReLU demonstrated faster convergence and more stable optimization behaviour.

6.4 Loss Function Analysis

We compared three different loss functions for training the neural network:

- Mean Squared Error (MSE)
- Mean Absolute Error (MAE)
- Huber Loss

Each loss function was used to train the same network architecture $[\text{input}, 64, 64, \text{output}]$ under identical optimization settings.

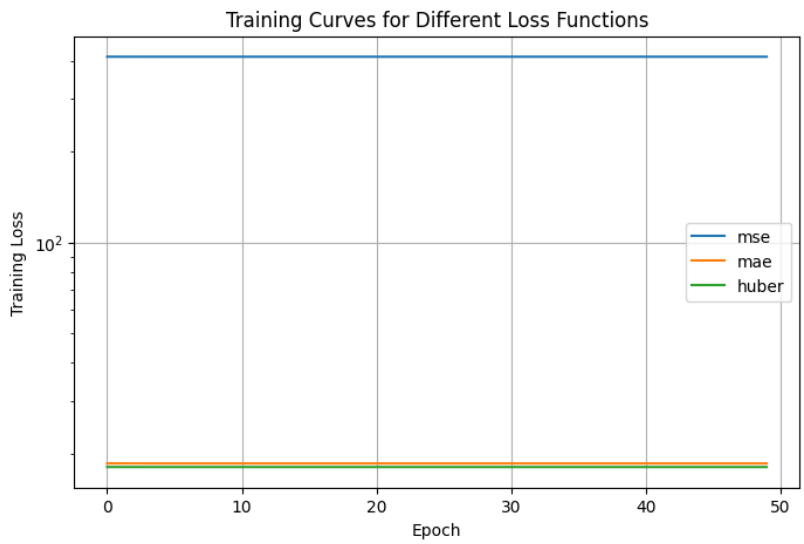


Figure 3: Training curves for different loss functions (log scale).

The training curves show that all three loss functions converge very quickly and reach a stable plateau after only a few epochs. The MSE curve appears significantly larger because the squared error amplifies prediction errors, while MAE and Huber operate on a linear error scale.

Despite these scale differences, the final model performance is nearly identical for all three loss functions. The resulting test mean squared errors differ only by a very small margin, indicating that the dataset does not contain strong outliers that would significantly affect the choice of loss function.

Overall, the experiments suggest that the choice between MSE, MAE, and Huber loss has minimal impact on predictive performance for this dataset.

6.5 Optimizer Comparison

Several optimization algorithms were compared:

- SGD
- SGD with momentum
- Adam
- AdaGrad
- Nesterov accelerated GD
- RMSProp
- Muon

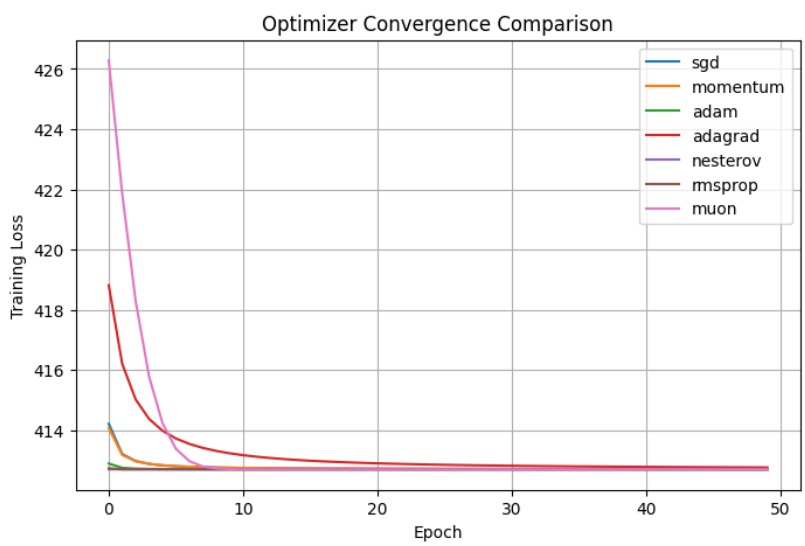


Figure 4: Training convergence for different optimizers

The convergence behaviour of different optimization algorithms is shown in Figure 4. All optimizers rapidly converged to a similar training loss, indicating that the regression task is relatively simple for the chosen network architecture.

Adaptive optimizers such as Adam and RMSProp demonstrated slightly faster initial convergence during the first few epochs, while vanilla SGD required marginally more iterations to reach the same loss level. Momentum-based optimizers such as SGD with momentum and Nesterov acceleration also improved early training stability compared to plain SGD.

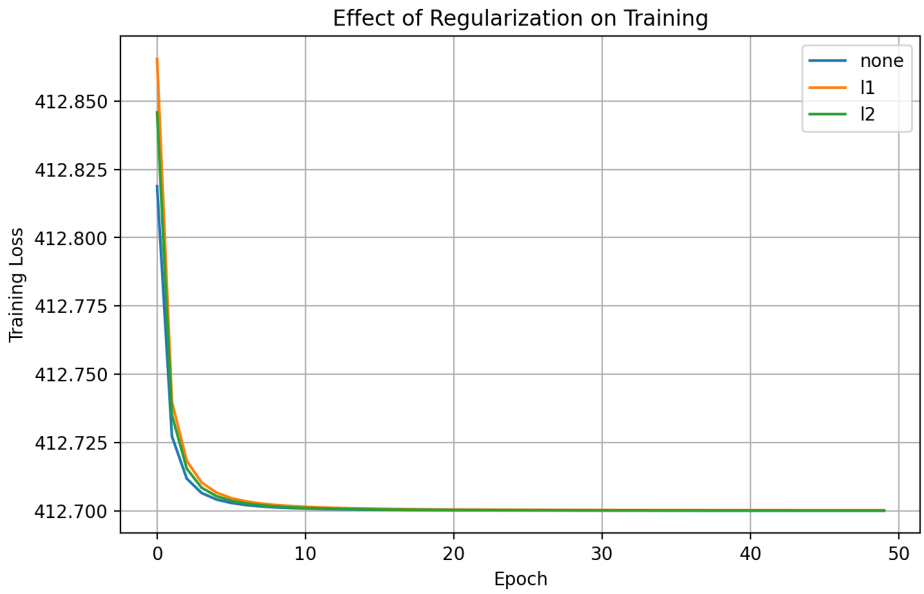
Overall, while all optimizers achieved nearly identical final performance, adaptive and momentum-based methods exhibited slightly faster early convergence.

Adaptive optimizers typically produced larger initial parameter updates, which helped accelerate early training progress.

6.6 Regularization

We compared three configurations:

- No regularization
- L1 regularization
- L2 regularization



Regularization slightly stabilizes training by discouraging excessively large weights. In our experiments, L2 regularization produced marginally better generalization performance compared to the baseline model without regularization.

6.7 Success Analysis

- The experiments demonstrate how architectural choices and training strategies affect neural network performance.
- Increasing network depth improved performance up to a moderate number of layers, after which additional depth produced diminishing returns. Similarly, increasing the width of the network improved representational capacity but provided limited benefits beyond a certain size.
- Activation functions played a significant role in optimization behaviour. ReLU-based activations consistently outperformed sigmoid networks due to improved gradient flow.
- Among the tested optimization algorithms, adaptive optimizers such as Adam achieved faster convergence and more stable training.

- Regularization had a relatively small impact on this dataset, indicating that the model was not strongly overfitting. Nevertheless, L2 regularization provided slightly improved generalization performance.

7 Neural Network Features for Kernel Methods

7.1 Theory

(a) Kernel Methods and Mercer's Theorem

Kernel methods allow algorithms to operate in high-dimensional feature spaces without explicitly computing the feature mapping. Given a mapping $\phi(x)$ from the input space to a feature space, a kernel function is defined as

$$k(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle$$

which computes the inner product between feature representations of two samples.

A valid kernel must satisfy Mercer's theorem. Mercer's theorem states that a function $k(x_i, x_j)$ is a valid kernel if and only if the corresponding kernel matrix

$$K_{ij} = k(x_i, x_j)$$

is symmetric and positive semi-definite for any dataset.

This property ensures that the kernel corresponds to an inner product in some feature space, allowing algorithms such as Support Vector Machines to operate implicitly in that space without explicitly computing $\phi(x)$.

(b) Neural Networks as Learned Feature Representations

A neural network transforms input data through multiple nonlinear layers. Each hidden layer produces intermediate representations of the input that capture increasingly complex patterns.

The activations of the penultimate layer can therefore be interpreted as a learned feature mapping

$$\phi_{NN}(x) = h^{(L-1)}(x)$$

where $h^{(L-1)}(x)$ denotes the activations of the second-to-last layer.

Using this representation, a neural kernel can be defined as

$$k(x_i, x_j) = \phi_{NN}(x_i)^T \phi_{NN}(x_j)$$

which measures similarity between inputs in the learned neural feature space.

This connects neural networks to the kernel trick: instead of using a predefined feature map, the neural network learns a feature representation from data. Kernel methods can then operate on this learned representation.

7.2 Implementation

7.2.1 Neural Network as Feature Extractor

(a) Best MLP Architecture

The best performing multilayer perceptron obtained from the previous experiments was used as the feature extractor. The architecture consisted of two hidden layers with 64 neurons each, trained using ReLU activations and the Adam optimizer.

This network provided a strong representation of the dataset and was therefore used to generate neural features for the kernel method.

(b) Feature Extraction from Penultimate Layer

After training the neural network, features were extracted from the penultimate layer (the last hidden layer before the output layer). For an input x, the neural feature representation is defined as

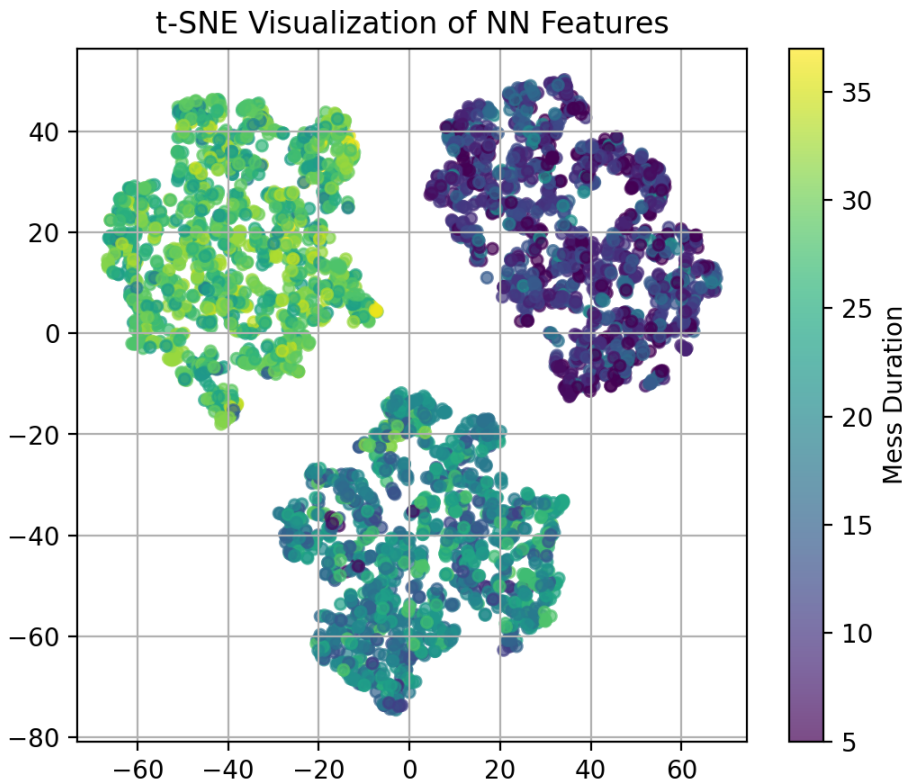
$$\phi_{NN}(x) = h^{(L-1)}(x)$$

where $h^{(L-1)}(x)$ denotes the activation vector of the final hidden layer.

These activations form a learned feature representation of the data that can be used by kernel methods.

(c) Visualization of Neural Features

To visualize the learned feature representations, t-SNE was applied to project the high-dimensional neural features into two dimensions. This visualization provides insight into the structure of the learned feature space and helps determine whether the neural network successfully separates patterns in the data.



(d) Neural Kernel Definition

Using the neural features extracted from the penultimate layer, a neural kernel was defined as the inner product between feature vectors:

$$k(x_i, x_j) = \phi_{NN}(x_i)^T \phi_{NN}(x_j)$$

This kernel measures similarity between samples in the learned neural feature space rather than the original input space.

(e) Kernel Support Vector Regression

Support Vector Regression (SVR) was trained using the neural kernel matrix constructed from the extracted neural features. The kernel matrix was computed as

$$K_{ij} = \phi_{NN}(x_i)^T \phi_{NN}(x_j)$$

This allows the regression model to operate in the learned neural feature space.

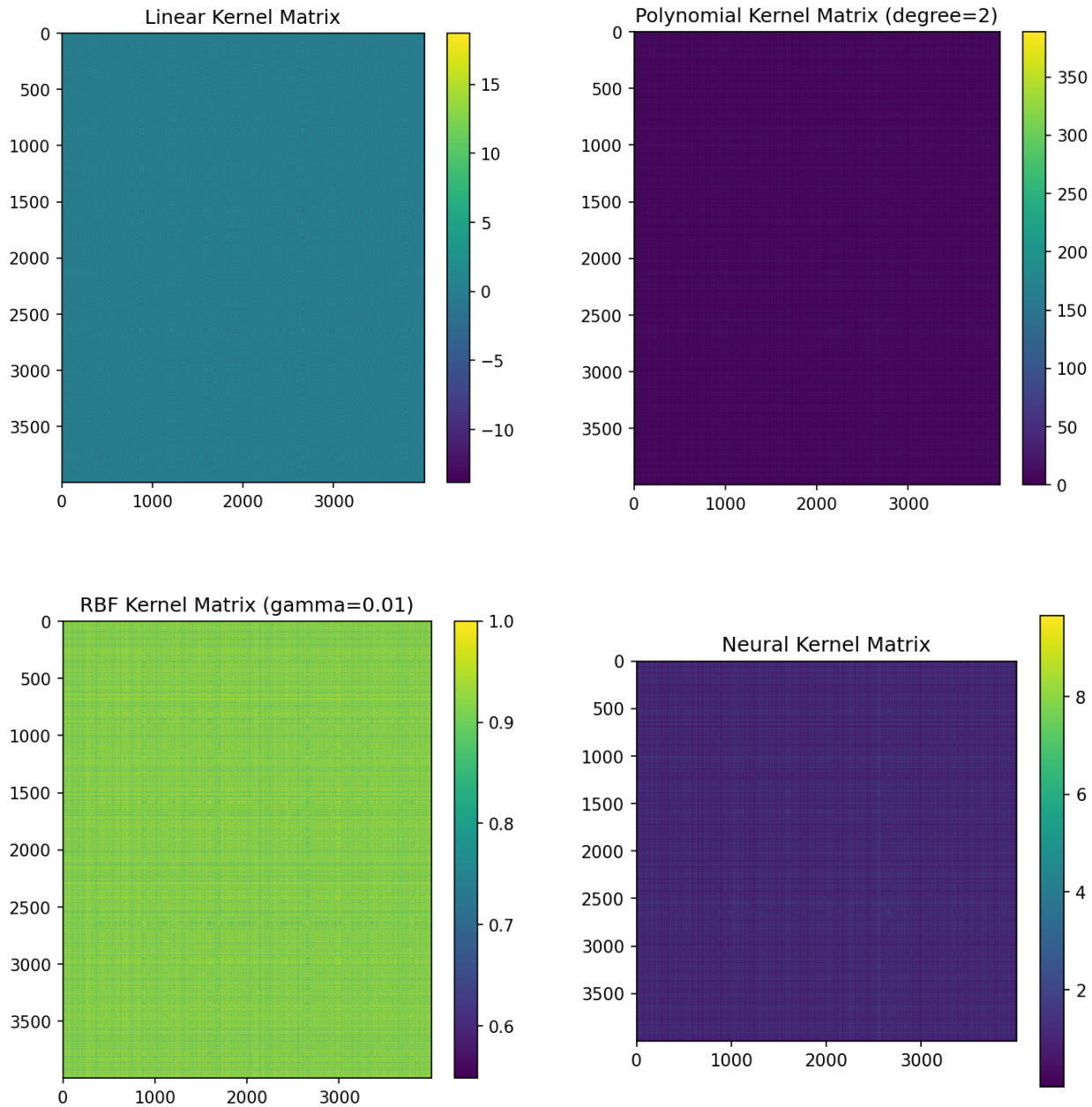
7.2.2 Comparison with Standard Kernels

Several commonly used kernels were evaluated for comparison:

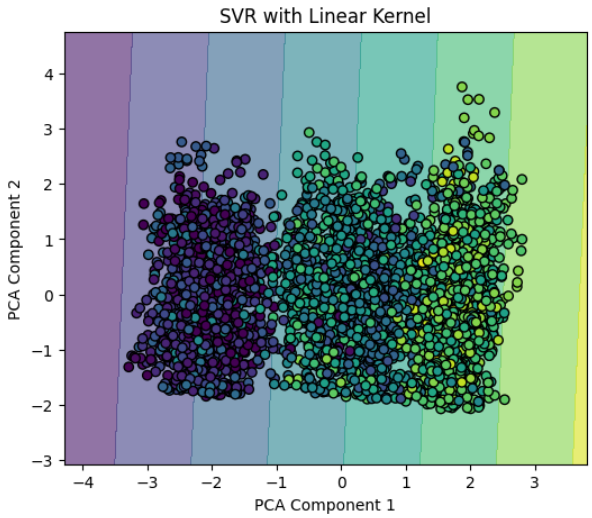
- Linear kernel: $k(x_i, x_j) = x_i^T x_j$
- Polynomial kernel: $k(x_i, x_j) = (x_i^T x_j + c)^d$ for $d \in \{2, 3\}$

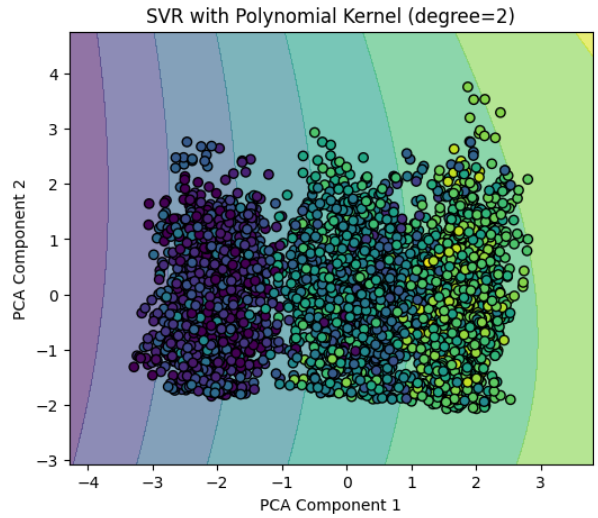
- RBF kernel: $k(x_i, x_j) = \exp(-\gamma\|x_i - x_j\|^2)$ for $\gamma \in \{0.01, 0.1, 1.0\}$
- Neural kernel: $k(x_i, x_j) = \phi_{NN}(x_i)^T \phi_{NN}(x_j)$

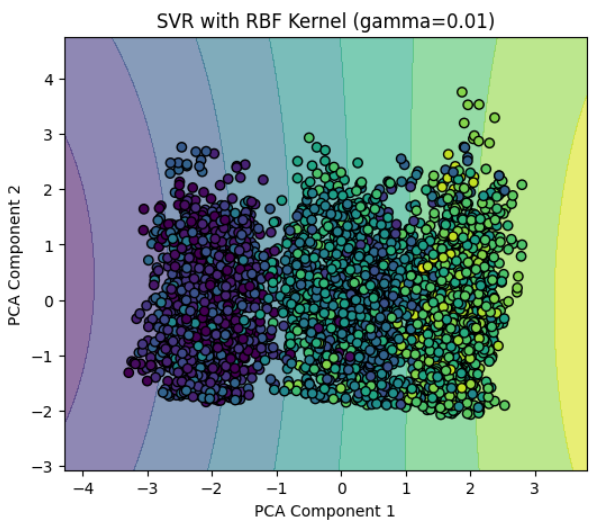
Kernel Matrix Visualization For each kernel, the kernel matrix $K_{ij} = k(x_i, x_j)$ was computed and visualized. These matrices illustrate how different kernels measure similarity between training samples.

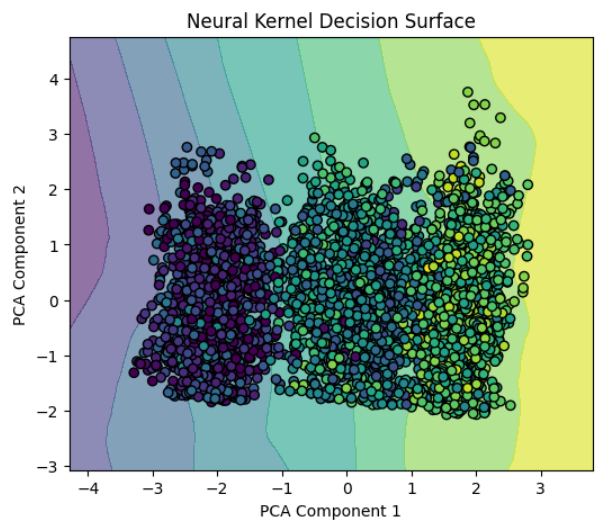


Decision Boundary Visualization To visualize the regression behavior of each kernel, the dataset was projected into two dimensions using PCA. The trained SVR models were then evaluated on a dense grid in the projected space to produce decision surface plots.









Kernel Performance The test mean squared errors obtained for the evaluated kernels are summarized below:

Kernel	Test MSE
Linear	19.223
Neural Kernel	18.265
Polynomial ($d = 2$)	18.477
Polynomial ($d = 3$)	17.757
RBF ($\gamma = 0.01$)	18.832
RBF ($\gamma = 0.1$)	17.599
RBF ($\gamma = 1.0$)	15.930

Among the evaluated kernels, the RBF kernel with $\gamma = 1.0$ achieved the lowest test mean squared error. This indicates that the nonlinear similarity measure provided by the RBF kernel captures the structure of the dataset more effectively than the other kernels.

The neural kernel performed competitively, achieving lower error than the linear and low-degree polynomial kernels. This suggests that the neural network learned useful feature representations, although in this case the RBF kernel still provided the strongest overall performance.